

01 Cloud architecture

A governed AWS landing zone with isolated environments and a repeatable Amazon EKS platform.

Landing zone and account boundaries

Use AWS Organizations and Control Tower for the account hierarchy, enrollment, and baseline controls. Reserve the management account for governance and billing. Separate Log Archive and Audit/Security accounts from Network, Observability, AFT management, Delivery, and AI tooling accounts. Group workload accounts in environment-specific OUs. AFT provisions and customizes accounts; separate pipelines build the EKS platform and applications. Additional security services and networking are explicitly configured, not assumed to be provided by Control Tower. [1,2]

Development through Production

Provide separate Development, Test, Staging, and Production accounts and EKS clusters per product/domain boundary. Do not put production into a shared nonproduction cluster. Within each cluster, namespaces, quotas, and RBAC organize trusted services; namespaces are not a hard tenant boundary. Dev uses synthetic data and short-lived previews; Test runs integration suites. Staging mirrors production versions, policies, and topology at lower capacity. Production uses approved releases, protected data, multi-AZ capacity, and tier-based recovery.

Infrastructure ownership and state

Version Terraform modules for accounts, VPCs, EKS, and application dependencies. Split state by account, environment, region, and manageable component; encrypt remote state and enforce locking, versioning, and scoped access. Plans may contain sensitive values and need short retention. Platform owns account/cluster lifecycle and approved add-ons; product teams own applications, data, SLOs, on-call, and costs. Drift detection opens a reviewable remediation change rather than blindly overwriting emergency fixes.

Kubernetes runtime and scaling

Use a private EKS API and EC2 workers across Availability Zones. Keep system controllers on an On-Demand managed node group; use bounded Karpenter NodePools for application capacity. HPA adjusts pod replicas while Karpenter provisions its own nodes. Define requests/limits, topology spread, readiness/startup probes, disruption budgets, and capacity headroom. PodDisruptionBudgets constrain voluntary disruption; they do not prevent AZ failure. Validate Kubernetes, node-image, and add-on compatibility through Staging before controlled production upgrades. [3]

Data availability and disaster recovery

Use Aurora with a writer and failover-capable reader across AZs, plus encrypted S3 object storage. Critical services have a separate regional EKS cluster, available images/manifests/configuration/secrets, Aurora Global Database replication, and cross-account backups. Recovery promotes the database, scales capacity, and switches application traffic through a rehearsed runbook. Replication is not a backup. Validate region/engine support, replication lag, restore behavior, and data consistency; a regional outage can cause data loss within the agreed RPO. [4]

Assumptions and trade-offs

This design assumes containerized web/API workloads. Select primary and recovery regions using residency, latency, service availability, and cost requirements. Example critical-tier targets are SLO 99.9%, RTO 60 minutes, and RPO 15 minutes; business owners must approve them and tests must demonstrate them. Separate clusters strengthen isolation but increase operating and control-plane costs. Reassess product/domain boundaries without weakening production isolation. Regional DR is funded by service criticality rather than applied identically to every workload.

Acceptance: account vending and repeatable rebuild succeed; AZ-loss, backup restore, and regional recovery exercises meet the agreed service-tier targets.

[1] Control Tower account structure | [2] AFT scope | [3] EKS reliability | [4] Aurora Global Database

02 Security controls

Layer organization guardrails, workload identity, Kubernetes policy, and independent security monitoring.

Human access and production elevation

Federate the corporate identity provider through IAM Identity Center with MFA and team-based permission sets. Map approved roles to EKS access entries and scoped Kubernetes RBAC; routine developers do not receive cluster-admin. Production access is read-only where justified; elevation requires an independent approver, purpose, expiry, and audit. Use tested break-glass access for identity outages. Review access regularly and remove it on role changes. Application authorization remains mandatory even when the network is private. [5]

Workload and delivery identities

Assign EKS Pod Identity roles to individual service accounts and environments; install the required agent and constrain role trust and resource permissions. Separate node, controller, application, infrastructure-deploy, and GitOps identities. Prevent ordinary pods from obtaining node-role credentials through IMDS. External CI uses tightly scoped OIDC federation; AWS-hosted tooling uses short-lived role sessions. Never expose deployment credentials or privileged runners to untrusted PR code. Application identities cannot change IAM, cluster access, or policy exceptions. [6]

Secrets, encryption, and data protection

Store secrets in Secrets Manager and retrieve them through workload identity, using SDKs or an approved CSI integration. Rotate secrets with tested application reload behavior. Keep secret values out of source control, images, Terraform inputs/state where possible, and logs. Use TLS, encrypted volumes/storage, and KMS keys with narrowly scoped policies. Select retention and backup controls by data classification. If Kubernetes Secrets are required, restrict read access and explicitly verify the cluster encryption configuration and key-management requirements.

Organization and Kubernetes guardrails

Apply SCPs and Control Tower controls to restrict unauthorized regions, public exposure, and changes to security baselines. SCPs limit maximum permissions and do not grant access or apply to the management account. Proactive CloudFormation controls do not replace Terraform policy checks. Enforce restricted Pod Security Admission, default-deny NetworkPolicy with a supporting CNI, and admission checks for trusted registries and verified image signatures. Deny privileged containers, unnecessary host access, and unapproved service-account use. [7,8]

Software supply chain and detection

Require protected branches, independent review, secret scanning, SAST/SCA, image/laC scans, SBOMs, and signed immutable releases. Block exploitable critical findings unless an accountable, time-limited exception is approved. Configure GuardDuty, Security Hub, Inspector coverage, CloudTrail, and EKS audit collection with delegated administration where supported. Findings route to named owners with remediation targets. Validate telemetry coverage explicitly; installing a service is not evidence that every account, region, or workload is monitored.

Exceptions and incident containment

Security owns the baseline and exception process; platform automates enforcement; product teams remediate workload findings. Exceptions record scope, reason, compensating controls, owner, and expiry. Protect audit storage against tampering and validate retention requirements before using irreversible retention settings. During an incident, isolate affected workloads/accounts, restrict credentials and network paths, and preserve evidence. Test denial cases, session expiry, break-glass access, and detection delivery; do not rely only on successful authorized access tests.

Acceptance: unauthorized role escalation, cross-environment secret access, privileged pods, unsigned images, and expired production elevation are rejected and auditable.

03 Networking approach

Private application tiers, controlled cross-account connectivity, and explicit traffic paths.

Regional network foundation

Use the Network account for IPAM, Transit Gateway (TGW), Route 53 Resolver, and inspected egress. Allocate nonoverlapping CIDRs before account onboarding. Separate production and nonproduction TGW route tables and attachments; permit only reviewed shared-service flows. OU membership or account enrollment does not create network trust. Avoid default full-mesh connectivity and introduce cross-region routing only for a documented dependency. Network changes follow reviewed IaC, route validation, and rollback planning.

Public ingress to private pods

Resolve application DNS through Route 53 to CloudFront, with AWS WAF and ACM-managed certificates. CloudFront reaches an internal ALB through a VPC origin. AWS Load Balancer Controller configures the Kubernetes Ingress and registers pod IP targets. Restrict origin security groups to the documented CloudFront path and allow only required ALB-to-pod ports. The controller manages load-balancer configuration; it is not an extra application proxy hop. API authorization and rate limits remain application/platform responsibilities. [9,10]

Private API and operator connectivity

Disable public EKS API access and provide private runners/operators with approved VPN, Direct Connect, or other private connectivity and DNS resolution. Enforce IAM and Kubernetes authorization independently of network access. Allow necessary control-plane, webhook, node, and health-check flows. For VPC origins, validate supported regions/protocols and the documented internet-gateway prerequisite; the private origin subnet does not require an internet route. A private endpoint alone is neither authentication nor authorization. [9]

Subnet and workload segmentation

Separate private worker subnets from isolated database subnets and allocate sufficient VPC CNI pod IP capacity for scaling and rolling updates. Use topology-aware scheduling across AZs. Apply Kubernetes NetworkPolicy for pod/namespace isolation and security groups for AWS resource boundaries; add security groups for pods only where their operating model is justified. Explicitly allow DNS and required platform dependencies. Apply network policy through a supported, configured enforcement engine, not merely by creating policy objects. [11]

Egress, endpoints, and failure domains

Use private endpoints for supported AWS services such as ECR, S3, STS, and Secrets Manager when justified by access and cost needs. Provide the endpoints and DNS required by Pod Identity and cluster controllers. Route necessary internet traffic through resilient inspection/NAT paths with destination allowlists. Validate symmetric routing and firewall failover. Account for AZ-local capacity, cross-AZ transfer charges, endpoint hourly charges, and centralized-egress dependencies; a shared network failure must not silently disable all workloads.

DNS ownership and validation

Manage private hosted zones, resolver rules, and sharing centrally to avoid conflicting zones and split-horizon failures. Product teams own service records through approved automation. Review DNS query/flow logs for security and troubleshooting with bounded retention. Test CloudFront-to-pod reachability, cross-environment denial, private EKS access, endpoint resolution, external dependency failure, and IP exhaustion scenarios. Document ingress/egress ports and dependency owners, and include network reachability in preproduction release checks.

Acceptance: no direct public path to workers or databases; approved ingress/API paths work; nonproduction-to-production traffic and unapproved egress are blocked.

04 Observability strategy

Correlate customer outcomes, Kubernetes health, cloud changes, and AI activity without collecting unlimited data.

Telemetry architecture

Standardize application instrumentation on OpenTelemetry. Send Prometheus-format workload/platform metrics to Amazon Managed Service for Prometheus, logs to CloudWatch Logs, and traces through a supported OpenTelemetry pipeline to AWS X-Ray. Amazon Managed Grafana provides curated views across approved data sources; CloudWatch supplies native AWS service metrics and alarms. Deploy collectors with resource limits, buffering/retry limits, and health monitoring. Assign each signal a primary destination to avoid duplicate ingestion and inconsistent alerts. [12]

Monitoring and service indicators

Track request rate, error rate, latency, saturation, and business success rate. For EKS, monitor pending pods, restarts, OOM events, node pressure, available replicas, HPA/Karpenter behavior, API errors, and subnet IP capacity. For Aurora, track connections, latency, storage, failovers, and replication lag. Define user-facing SLIs and SLOs with product owners; use error-budget consumption to inform release decisions. Infrastructure health is supporting evidence, not a substitute for customer transaction success.

Logs and distributed tracing

Collect structured JSON application logs with service, environment, version, owner, and trace/correlation IDs. Enable EKS control-plane/audit logs and centralize organization audit records separately from operational logs. CloudTrail records AWS activity; Kubernetes API actions require EKS audit logs. Propagate trace context across ingress, services, queues, and database calls where supported. Sample traces by value and risk; retain sufficient failure evidence without logging credentials, personal data, or full request payloads by default. [12]

Alerting and dashboards

Page the owning on-call team for actionable customer-impact symptoms and sustained SLO burn; route capacity trends and low-urgency findings to tickets. Use multiple evaluation windows, deduplication, dependency context, and maintenance handling to reduce noise. Every alert includes service owner, severity, dashboard, and runbook. Provide service dashboards for teams, cluster/fleet dashboards for platform operations, and SLO/cost summaries for management. Overlay deployments and configuration changes to support causal investigation.

Retention, access, and cost controls

Set retention by signal and data classification. Starting assumptions are 30 days of searchable application logs and longer security retention in an encrypted archive, subject to compliance approval. Configure protected/immutable audit storage and lifecycle transitions explicitly. Scope cross-account observability roles to approved data and separate production from nonproduction access. Bound metric cardinality, scrape frequency, log verbosity, and tracing volume. Alert on collector drops and ingestion failures; sampling must not silently remove required security audit records. [13]

Visibility into AI-driven activity

Correlate requester, agent identity, task ID, repository/commit, approval, tool invocation, target resource, and result. Record model/version, token usage, duration, retries, and cost, while redacting sensitive content. Combine gateway audit with CloudTrail, EKS audit, and pipeline logs; no single source captures the full task. Dashboards show completion rate, denied actions, unusual permissions/targets, repeated failures, and cost per successful task. Record structured decisions and outcomes rather than private model reasoning.

Acceptance: a synthetic failed transaction and a denied AI operation can be traced end to end; alerts reach the correct owner; telemetry loss is detected.

05-06 Developer experience and cost management

Make the approved path easy to use and give each team ownership of measurable unit costs.

Self-service through approved templates

Provide an internal developer portal with a service catalog, owners, runbooks, SLOs, dependencies, and cost links. Templates create a repository, container build, Terraform module references, Helm/Kustomize configuration, Pod Identity, quotas, policies, and dashboards. Engineers request accounts or platform services through PRs with cost/data metadata. Standard requests proceed automatically after policy checks; new regions, privileged access, or nonstandard exposure require review. Document supported patterns and a time-bounded exception path.

Deployment and safe service consumption

Build once and publish signed images/SBOMs to ECR. Promote the same digest through Dev, Test, Staging, and Prod. Scoped GitOps controllers reconcile approved, versioned manifests; application deployment cannot change cluster-wide security controls. Run tests and scans before promotion and use protected production gates. Track rollout health and revert the release when SLO checks fail. Database migrations use backward-compatible expand/contract; reverting a manifest does not reverse data changes. Provide versioned service contracts and least-privilege access modules.

Adoption, support, and delivery metrics

Offer working examples, local-development guidance, preview environments, office hours, and clear support ownership. Measure time to first deployment, provisioning lead time, deployment frequency, change failure rate, recovery time, and developer satisfaction. Publish platform SLOs and maintenance notices. Template and add-on upgrades include migration guidance and a deprecation window. Use team feedback and support-ticket patterns to prioritize platform work; self-service succeeds when teams can complete routine work without a platform ticket.

Allocation and ongoing cost controls

Export AWS cost/usage data to S3 and analyze it with Athena/Cost Explorer. Activate owner, service, environment, and cost-center allocation metadata; use Cost Categories for shared platform costs. Enable EKS split cost allocation for pod/namespace compute attribution, with an explicit method for idle capacity and remaining shared costs. Review cost per transaction/customer and per successful AI task. Budgets and anomaly detection notify owners; enforce quotas, expiry, and admission limits separately. [14,15]

Investigating a 40% rise over three months

Normalize month length, currency, credits, and amortized commitments. Break down the increase by account, service, environment, region, and usage type, then correlate it with traffic and releases. Review EC2/node utilization, pod requests versus actual usage, idle clusters, EKS support charges, storage/IOPS, NAT/egress, log ingestion, and AI token/retry volume. Compare unit costs: 40% higher spend with 80% more transactions means approximately 22% lower cost per transaction. Assign owners to the largest unexplained increases.

Optimization and risk assessment

Remove orphaned resources and expired previews first. Then rightsize requests/nodes, tune HPA and Karpenter, reduce low-value telemetry, and use Spot for interruption-tolerant workloads. Buy commitments only after baseline demand is understood and optimized. Model the cost of separate clusters and DR explicitly. Do not reduce backup retention or availability headroom without SLO/RTO/RPO review. Validate each change through load/failure testing, staged rollout, and measured savings; retain rollback capacity and report any reliability trade-off.

Acceptance: a team can onboard and deploy through the portal; every service has an owner/cost allocation; optimization savings are verified against SLOs.

[14] EKS split cost allocation | [15] AWS Cost Anomaly Detection

07 AI governance controls

Agents propose and execute bounded tasks; identity, approval, and enforcement remain outside the model.

Identity and execution isolation

Run engineering agents in the dedicated AI tooling account. Register each agent with an owner, purpose, approved tools/models, data classification, and review/expiry date. Authenticate with a dedicated workload role or trusted federation and issue short-lived, task-scoped sessions. Preserve the requesting human identity separately. Execute each task in an isolated sandbox with resource/time limits and controlled egress. Agents do not inherit a user session, long-lived administrator key, or the CI/CD production identity.

Authorization and tool permissions

A tool gateway and policy engine validate every request against agent identity, task scope, environment, resource, operation, and parameters. IAM, Kubernetes RBAC, admission policy, and repository protections enforce the result. Agents may generate code, create PRs, and propose infrastructure changes. Allow operational execution only through versioned, allowlisted runbooks with validated inputs and bounded effects. Deny privilege escalation, self-approval, policy changes, credential export, and arbitrary production shell access. Model output cannot create new authority.

Human oversight and approval binding

Preapprove low-risk read-only or reversible nonproduction tasks within explicit limits. Require independent human approval for production changes, destructive/data operations, and new sensitive access. Bind approval to task ID, artifact/commit digest, infrastructure plan or runbook version, exact target, parameters, and expiry. Any material change invalidates approval. Recheck authorization at execution time, serialize conflicting operations, and fail closed if policy/approval services are unavailable. Approved deterministic rollback runbooks can execute within their already authorized scope.

Data handling and prompt-injection defense

Treat repository content, documents, retrieved text, and tool responses as untrusted data, never as permission instructions. Separate instructions from content and validate tool arguments/output schemas. Restrict model endpoints, data access, outbound destinations, and repository scope; redact secrets and personal data. Evaluate model/provider retention, training-use, and regional-processing terms before enabling sensitive inputs. Prompt-attack detection or Bedrock Guardrails is an additional signal, not a guarantee or an authorization boundary. Test adversarial content and exfiltration attempts. [16]

Auditability and quality evaluation

Maintain tamper-resistant records of requester, agent/task identity, model/version, approval evidence, policy decision, tool calls, resource changes, outcome, and cost. Link PRs and deployments to the same task; collect denied and failed actions too. Limit access to sensitive prompts and outputs and apply retention rules; do not collect private chain-of-thought. Test code quality, runbook safety, authorization denials, prompt injection, and budget exhaustion before onboarding or upgrading agents. Owners review anomalies, permission drift, and access periodically.

Emergency suspension and recovery

Provide an independently operated kill switch that denies new tool actions, cancels queued/running jobs, and isolates sandbox networking. Block new role assumptions and revoke supported active role sessions or apply explicit denies; removing trust alone does not invalidate every issued credential. AWS authorization changes can propagate asynchronously, so gateway/job/network containment is the immediate control. Rotate exposed secrets, preserve evidence, and assess affected changes. Reactivation requires owner/security review and a limited validation run. [17]

Acceptance: injected instructions cannot bypass policy; changed targets invalidate approval; a suspension drill stops tool execution and leaves a complete audit trail.